

RUSTUP & CARGO

rustup update | default stable | toolchain install
nightly
rustup target add wasm32-wasip1 | component add clippy
rustfmt
cargo new shelf [--lib] | init | add serde -F derive
cargo build [--release] | run -- args | check | test |
bench
cargo doc --open | fmt | clippy -- -D warnings | tree
| update
cargo +nightly miri test | cargo install ripgrep

CARGO.TOML SKELETON

```
[package]
name = "shelf-core"    version = "0.1.0"
edition = "2024"      rust-version = "1.85"
[dependencies]
serde = { version = "1", features = ["derive"],
optional = true }
[features]
default = []    json = ["dep:serde"]
[profile.release]
lto = "fat"    codegen-units = 1    debug = "line-tables-only"
[lints.rust]    unsafe_code = "forbid"
[lints.clippy]    pedantic = { level = "warn", priority = -1 }
# workspace root: {workspace} members = ["crates/*"]
# resolver = "3"; {workspace.dependencies} tokio = "1"
# member: tokio = { workspace = true }
```

PRIMITIVE TYPES, LITERALS, CASTS

Ints

i8..i128, isize; u8..u128, usize · float f32 f64 · bool · char (4-byte Unicode scalar) · () unit · ! never
1_000u32 0xFF 0o17 0b1010 b'a' 1e-3_f64 'é' "s"
r#"raw"# b"bytes" c"cstr"
let t: (i32, &str) = (1, "a"); t.0 let a = [0u8; 4];
x as u8 // truncates/saturates (float-int)
u8::from(b) / i32::try_from(n)?
n.checked_add(1) wrapping_mul saturating_sub
overflowing_add strict_add (1.91)

VARIABLES

```
let x = 5; // immutable by default
let mut n = 0; n += 1;
let x = x * 2; // shadowing: new binding, may
change type
const MAX: usize = 64; // inlined, any scope
static GREETING: &str = "hi"; // one address,
'static
let (a, mut b) = (1, 2); let Book { title, .. } =
book;
```

CONTROL FLOW

```
let v = if ok { 1 } else { 0 }; // if is an
expression
let n = loop { if done() { break 42; } }; // loop
returns a value
'outer: for i in 0..10 { for j in 0..i { continue
'outer; } }
while let Some(x) = stack.pop() { ... }
if let Some(b) = find(id) { ... } else { ... }
let Some(b) = find(id) else { return Err(NotFound) };
// let-else
if let Some(b) = find(id) && b.available { ... }
// let chains (2024)
match n { 0 => "zero", 1 | 2 => "few", 3..=9 =>
"some", _ => "many" }
```

OWNERSHIP & BORROWING RULES

1. Each value has **one owner**; when the owner goes out of scope the value is **dropped**.

2. Assignment / passing **moves** non-copy values; the source is unusable. `.clone()` to duplicate.

3. At any time: **either one &mut T or any number of &T**.

4. References must **never outlive** the value (no dangling); borrows end at last use (NLL).

Lifetime elision: (1) each elided input ref gets its own lifetime; (2) exactly one input lifetime → assigned to all outputs; (3) a &self/&mut self method → outputs get self's lifetime. Otherwise annotate:
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str
struct Excerpt<'a> { text: &'a str } T: 'static = owns no short borrows

STRUCTS, ENUMS, IMPL

```
#[derive(Debug, Clone, PartialEq)]
struct Book { isbn: ISBN, title: String, year: u16 }
struct ISBN(String); struct Marker; // tuple /
unit
enum LoanState { Available, OnLoan { member: u32 },
Lost(String) }
impl Book {
pub fn new(title: &str) -> Self { Self { title:
title.into(), ..Default::default() } }
pub fn title(&self) -> &str { &self.title }
fn rename(&mut self, t: String) { self.title = t;
}
fn into_title(self) -> String { self.title }
}
```

PATTERN SYNTAX

```
A | B alternatives 1..=9 'a'..'z'
ranges
Point { x, .. } / (first, ..., last) ignore rest
n @ 1..=12 bind and test Some(ref s) / &(a, b)
x if x > 0 match guard [head, tail @ ..]
slice
Some(x) if let Ok(n) = x.parse() => // if-let guard
(1.95)
```

OPTION, RESULT, ?

```
opt.map(f) .and_then(f) .filter(p) .unwrap_or(d)
.unwrap_or_default()
opt.ok_or(Err)? .ok_or_else(||...) .take() .as_ref()
.is_some_and(p)
res.map_err(Into::into)? .ok() .unwrap_or_else(|e|
...) .expect("why")
fn load(p: &Path) -> Result<Catalog, ShelfError> {
let text = fs::read_to_string(p)?; // ? = early
return + From::from
Ok(serde_json::from_str(&text)?) }
```

Libraries: thiserror enum per crate; apps:

```
anyhow::Result + .context("...")
#[derive(Debug, thiserror::Error)]
enum ShelfError { #error("no book {0}")
NotFound(Isbn), #error(transparent)) Io(#[from]
io::Error) }
```

COLLECTIONS & STRINGS

```
let mut v = vec![1, 2]; v.push(3); v.extend([4]);
v.retain(|x| x % 2 == 0);
v.sort_unstable_by_key(|b| b.year); v.dedup();
v.binary_search(&3);
let mut m = HashMap::new(); *m.entry(k).or_insert(0)
+= 1;
m.get(&k) m.remove(&k) BTreeMap (sorted) HashSet
VecDeque BinaryHeap
String (owned, UTF-8) vs &str (borrowed slice);
s.len() = bytes
format!("{}",title:>20) {n:05} {x:.2} {v:?} {v:#?}")
s.chars().bytes().split('') .lines().trim()
.to_lowercase()
s.parse::<u32>()? s.strip_prefix("x") &s[..3] (byte
idx, panics mid-char)
```

ITERATORS

```
iter() &T · iter_mut() &mut T · into_iter() T
Adapters (lazy): map filter filter_map flat_map
enumerate zip chain
take skip take_while step_by rev peekable windows
chunks inspect
Consumers: collect sum product count min_by_key max
fold any all
find position for each partition last nth
let titles: Vec<_> = books.iter().filter(|b| b.year <
1970)
.map(|b| b.title.as_str()).collect();
let m: HashMap<_, _> = books.into_iter().map(|b|
(b.isbn.clone(), b)).collect();
Result<Vec<_>, _> = iter.map(parse).collect(); //
stops at first Err
```

TRAITS & GENERICS

```
trait Catalog { fn find(&self, i: &ISBN) ->
Option<&Book>;
fn count(&self) -> usize { 0 } } //
default method
impl Catalog for InMemoryCatalog { ... }
fn a<C: Catalog>(c: &C) // static dispatch,
monomorphised
fn b(c: &impl Catalog) // same, shorter
fn c() -> impl Iterator<Item = u8> // opaque return
type
fn d(c: &dyn Catalog) // dynamic dispatch
via vtable
Vec<Box<dyn Catalog>> where T: Display + Clone
struct Buf<const N: usize>
Derivable: Debug Clone Copy PartialEq Eq PartialOrd
Ord Hash Default · Conversions: From/Into,
TryFrom/TryInto, AsRef, Borrow, FromStr,
Display→to_string, Deref
```

CLOSURES

```
let add = |a: i32, b| a + b; let owned = move ||
data.len();
FnOnce may consume captures (called once)
FnMut mutates captures (needs mut binding)
Fn only reads captures (callable concurrently)
fn apply<F: Fn(i32) -> i32>(f: F) Box<dyn FnMut()>
async || {...} (1.85)
```

SMART POINTERS & INTERIOR MUTABILITY

Need	Single thread	Multi-thread
heap, one owner	Box<T> (recursive types, dyn)	
shared ownership	Rc<T>	Arc<T>
break cycles	rc::Weak	sync::Weak
mutate Copy value	Cell<T>	AtomicU64 ...
mutate via &	RefCell<T> (runtime check)	Mutex / RwLock
init once	OnceCell / LazyCell	OnceLock / LazyLock

Typical: Rc<RefCell<T>> · Arc<Mutex<T>> · Cow<'_, str>
borrow-or-own

SEND & SYNC

Type	Send	Sync
i32, String, Vec, Box, Arc<Mutex>	✓	✓
Cell, RefCell, mpsc::Receiver	✓	✗
Rc, raw pointers	✗	✓
MutexGuard	✗	✓

Send: may move to another thread · Sync: &T is Send

THREADS, CHANNELS, ASYNC

```
let h = thread::spawn(move || work(data));
h.join().unwrap();
thread::scope(|s| { s.spawn(|| read(&v)); }); //
borrow locals
let (tx, rx) = mpsc::channel(); tx.send(x)?; for m in
rx { ... }
let n = Arc::new(Mutex::new(0)); *n.lock().unwrap() +=
1;
#[tokio::main] async fn main() { let b =
fetch(id).await; }
let (a, b) = tokio::join!(f1(), f2());
tokio::spawn(async move {...})
tokio::select! { v = rx.recv() => ..., _ = sleep(d) => ...
}
never block in async: tokio::task::spawn_blocking(||
cpu())
```

MODULES & VISIBILITY

```
mod catalog; // src/catalog.rs or
src/catalog/mod.rs
pub fn · pub(crate) · pub(super) · pub(in crate::a) ·
private default
use crate::catalog::{Catalog, InMemoryCatalog as Mem};
use super::*; self:: pub use model::Book; // re-
export
lib.rs = library root, main.rs = binary root,
src/bin/*.rs
```

MACRO RULES! & UNSAFE

```
macro_rules! shelf { ($<expr>,* $<(),?) => { vec!
[$(Book::new($t)),*] }; }
```

Fragments: expr ident ty pat pat_param path stmt
block item literal lifetime meta vis tt · repeat \$(...),*
+ ?

unsafe superpowers: deref raw pointers · call unsafe
fn/FFI · access/modify static mut · implement unsafe
trait · access union fields. Document every block with
// SAFETY:..

TESTING & TOOLING

```
#[cfg(test)] mod tests { use super::*;
#[test] fn ok() { assert_eq!(f(2), 4); }
#[test] #[should_panic(expected = "empty")] fn p()
{}
#[test] #[ignore] fn slow() {} fn r() -> Result<(),
E> }
/// `` doc test ``no_run ``ignore
``should_panic ``compile_fail
cargo test name -- --nocapture --ignored tests/*.rs
= integration
cargo clippy --all-targets -- -D warnings cargo fmt
--check
#[expect(clippy::lint, reason = "...")] #[must_use] #
[deprecated]
```

Changed since older tutorials

2024 edition (1.85): #
[unsafe(no_mangle)], unsafe extern "C" {...}

wasmb32-wasi → wasmb32-wasip1 / wasip2
(1.84)

#[async_trait] → native async fn in traits
(1.75)

trait upcasting &dyn Sub → &dyn Super
(1.86)

object safety → dyn compatibility
(renamed 2024)

let chains in if / while (1.88, edition
2024)

lazy_static! / once_cell → LazyLock / OnceLock
(1.80/1.70)

#[allow] → #[expect] (1.81);
assert_matches! (1.96)